

PROJECT PROGRESS REPORT

SOFTWARE SYSTEMS DESIGN PROJECT

CO2060

Group No: E22 GR23

AI SMART ASSISTANT

Department of Computer Engineering

Faculty of Engineering

University of Peradeniya

2026

Group Details:

Group No: E22 GR23

No.	Name of the Student	Index Number	E-mail address	Signature
1	K.J.V. Kahagalla	E/22/179	e22179@eng.pdn.ac.lk	
2	K.A.H. Kumarasinghe	E/22/205	e22205@eng.pdn.ac.lk	
3	P.A.K.N. Sooriyabandara	E/22/378	e22378@eng.pdn.ac.lk	
4	K.D.N. Chandrasena	E/22/054	e22054@eng.pdn.ac.lk	

Contents

1	Introduction	1
1.1	Project Title	1
1.2	Project Description	1
1.3	Background and Motivation	1
1.4	Problem in Brief	1
1.5	Proposed Solution	2
1.6	Project Aim and Objectives	2
1.7	Significance of the study	2
2	Methodology	3
2.1	Introduction	3
2.2	Requirements Identification	3
2.2.1	Functional and Non-functional requirements	3
2.2.2	User roles	3
2.2.3	System requirements (Hardware / Software)	4
2.3	System Analysis and design	4
3	System Implementation and Progress	5
3.1	Introduction	5
3.2	Current Implementation Status	5
3.2.1	Database Realization	5
3.2.2	Backend and Core Logic	5
3.2.3	Frontend and User Interface	5
3.3	Functional Milestones Achieved	6
3.4	Technical Challenges and Deviations	6
4	Project Plan	7
4.1	Project Plan	7
4.2	Individual contribution	7
4.3	Future Work	7

Chapter 1

Introduction

1.1 Project Title

AI Smart Assistant

1.2 Project Description

The AI Smart Assistant is an intelligent, privacy-first desktop software application designed to assist non-technical users in troubleshooting computer errors. It operates entirely offline, utilizing a combination of screen-reading Optical Character Recognition (OCR) and a locally hosted Large Language Model (Mistral via Ollama) to analyze on-screen errors and provide actionable, step-by-step resolution instructions through a custom chat interface.

1.3 Background and Motivation

Modern operating systems and software applications frequently present cryptic error messages that are difficult for average users to decipher. Typically, resolving these issues requires manual internet searches, navigating forums, or contacting IT support. Our motivation for this project stems from the need to provide instant, accessible troubleshooting directly on the user's screen without compromising their data privacy by sending screen captures to cloud-based AI services.

1.4 Problem in Brief

The existing system of troubleshooting (manual searching) is slow, frustrating for non-technical users, and often ineffective. Furthermore, relying on modern cloud AI assistants (like ChatGPT or Copilot) introduces significant privacy risks, as users must upload screenshots of their desktop, which may contain sensitive personal or corporate data.

1.5 Proposed Solution

We propose a lightweight, background-running desktop assistant. Using a global hotkey, the user can draw a bounding box around any error message on their screen. The system uses local OCR (Tesseract) to read the text, queries a local AI model (Ollama/Mistral) for a solution, and displays the fix in a floating chat window ensuring 100% offline data privacy.

1.6 Project Aim and Objectives

Aim: To build a fully autonomous, offline AI troubleshooting assistant for Windows.

Objectives:

- Develop an accurate, high-speed OCR pipeline for screen text extraction.
- Integrate a local LLM API capable of streaming text responses.
- Build a custom, asynchronous graphical user interface (GUI) that does not disrupt the user's workflow.
- Deploy the application as a standalone Windows installer.

1.7 Significance of the study

Compared to existing systems like Microsoft Copilot, which require constant internet connectivity and upload telemetry/screen data to remote servers, the AI Smart Assistant guarantees absolute data privacy. It empowers average users to fix their own computers instantly without relying on external IT support, reducing downtime and frustration.

Chapter 2

Methodology

2.1 Introduction

The project follows an Agile software development life cycle. Requirements were gathered by interviewing potential end-users (both technical and non-technical) to understand their pain points with computer errors. Development is carried out in iterative sprints, starting with a Minimum Viable Product (MVP) focused on basic OCR-to-AI communication, followed by UI refinement and deployment packaging.

2.2 Requirements Identification

2.2.1 Functional and Non-functional requirements

Functional Requirements:

- The system must capture a user-defined region of the screen via a hotkey.
- The system must extract text from the captured image accurately.
- The system must query an offline LLM and return a troubleshooting solution.
- The system must cache previous solutions to speed up response times.

Non-functional Requirements:

- **Privacy:** All data processing must occur locally (0% cloud dependency).
- **Performance:** The OCR extraction must take less than 500ms.
- **Usability:** The UI must be intuitive and non-intrusive (borderless overlay).

2.2.2 User roles

The primary user role is the **End-User**, who interacts with the client interface to resolve PC errors.

2.2.3 System requirements (Hardware / Software)

Software: Windows 10/11, Python 3.10+, Tesseract OCR Engine, Ollama (Mistral model).

Hardware: Minimum 8GB RAM and a multi-core CPU. A dedicated GPU is recommended for faster LLM inference, though CPU fallback is supported.

Justification: Python was selected for its robust libraries (`mss`, `pytesseract`, `tkinter`). Local Ollama was chosen specifically to satisfy the strict data privacy non-functional requirement.

2.3 System Analysis and design

(Please refer to Appendix for System Architecture Diagrams). The system utilizes a multi-processing architecture. A background listener process monitors for global hotkeys and manages the OCR/AI pipeline, while a separate process handles the Tkinter GUI rendering to prevent application freezing.

Chapter 3

System Implementation and Progress

3.1 Introduction

This section outlines the current developmental state of the AI Smart Assistant at the Milestone 3 reporting phase. It highlights the successful integration of the core backend modules with the frontend GUI and the successful deployment packaging.

3.2 Current Implementation Status

3.2.1 Database Realization

A custom NoSQL-style JSON database (`errors_db.json`) has been successfully implemented. It caches the OCR text and the corresponding AI solution. We integrated a fuzzy-matching algorithm (using Python's `difflib`) to retrieve cached solutions even if the OCR misreads a single character, significantly improving response times.

3.2.2 Backend and Core Logic

The core algorithmic components are fully operational. The `mss` screen capture pipeline is integrated with Tesseract OCR. The image pre-processing pipeline (using PIL/Pillow) has been optimized with contrast enhancements and Lanczos resampling to improve text recognition on high-DPI displays. The API communication with the local Ollama server has been restructured to support chunked streaming responses.

3.2.3 Frontend and User Interface

A custom, borderless Tkinter user interface has been constructed. It features dynamic text streaming (a "typing" effect), smooth polygon-based chat bubble animations, and an interactive settings panel for hotkey configuration. The UI correctly handles multi-monitor scaling.

3.3 Functional Milestones Achieved

- **Requirement 01: Screen Capture & OCR** - Status: Fully Implemented
- **Requirement 02: Local AI Integration** - Status: Fully Implemented
- **Requirement 03: Asynchronous UI** - Status: Fully Implemented
- **Requirement 04: Windows Installer Deployment** - Status: Fully Implemented

3.4 Technical Challenges and Deviations

A major technical challenge encountered was integrating the OCR output with the caching database. Initially, the logic required a 100% exact text match, which failed frequently due to minor OCR typos (e.g., misreading 'O' as '0'). To mitigate this without compromising the core project scope, we adapted the workflow by implementing a fuzzy-matching sequence algorithm, which successfully resolves minor typos dynamically.

Chapter 4

Project Plan

4.1 Project Plan

The initial sprints focused on establishing the OCR and AI pipeline (MVP). Mid-semester efforts were reallocated towards system deployment, specifically engineering a standalone Windows Installer using Inno Setup to ensure all dependencies (Python, Tesseract) are bundled. The final sprints will focus on UI markdown rendering and final User Acceptance Testing.

4.2 Individual contribution

The workload was divided as follows:

- **K.J.V. Kahagalla:** UI/UX Engineering (Tkinter redesign, streaming text frontend, animations).
- **K.A.H. Kumarasinghe:** OCR & Display Optimization (multi-monitor support, high-DPI fixes, image pre-processing).
- **P.A.K.N. Sooriyabandara:** AI Integration (Ollama streaming API, prompt engineering, output formatting).
- **K.D.N. Chandrasena:** Deployment & Architecture (Inno Setup Windows Installer, fuzzy-matching database caching).

4.3 Future Work

- **K.J.V. Kahagalla:** Integrate a full markdown rendering engine within the Tkinter chat window.
- **K.A.H. Kumarasinghe:** Conduct performance profiling on Tesseract to further reduce latency.

- **P.A.K.N. Sooriyabandara:** Implement fallback mechanisms for slower hardware (e.g., swapping to a smaller LLM model dynamically).
- **K.D.N. Chandrasena:** Conduct installation testing on clean virtual machines and finalize system documentation.